

11 Transactions and concurrency

This chapter covers

- Defining database and system transaction essentials
- Controlling concurrent access with Hibernate and JPA
- Using non-transactional data access
- Managing transactions with Spring and Spring Data

In this chapter, we'll finally talk about transactions: how we create and control concurrent units of work in an application. A *unit of work* is an atomic group of operations, and transactions allow us to set unit of work boundaries and help us isolate one unit of work from another. In a multi-user application, we may also be processing these units of work concurrently.

To handle concurrency, we'll first focus on units of work at the lowest level: database and system transactions. You'll learn the APIs for transaction demarcation and how to define units of work in Java code. We'll demonstrate how to preserve isolation and control concurrent access with pessimistic and optimistic strategies. The overall architecture of the system affects the scope of a transaction; a bad architecture may lead to fragile transactions.

Then we'll analyze some special cases and JPA features, based on accessing the database without explicit transactions. Finally we'll demonstrate how to work with transactions with Spring and Spring Data.

Let's start with some background information.

Major new features in JPA 2

There are new lock modes and exceptions for pessimistic locking:

- You can set a lock mode, pessimistic or optimistic, on a `Query`.
- You can set a lock mode when calling `EntityManager#find()`, `refresh()`, or `lock()`. A lock timeout hint for pessimistic lock modes is also standardized.

When the new `QueryTimeoutException` or `LockTimeoutException` is thrown, the transaction doesn't have to be rolled back.

The persistence context can now be in an *unsynchronized* mode with disabled automatic flushing. This allows us to queue modifications until we join a transaction and to decouple the `EntityManager` usage from transactions.

11.1 Transaction essentials

Application functionality requires that several things be done in one go. For example, when an auction finishes, the `CaveatEmptor` application must perform three different tasks:

1. Find the winning bid (highest amount) for the auction item.
2. Charge the seller of the item the cost of the auction.
3. Notify the seller and successful bidder.

What happens if we can't bill the auction costs because of a failure in the external credit card system? The business requirements may state that either all listed actions must succeed or none must succeed. If so, we call these steps collectively a *transaction* or *unit of work*. If only a single step fails, the entire unit of work must fail.

11.1.1 ACID attributes

ACID stands for *atomicity*, *consistency*, *isolation*, *durability*. *Atomicity* is the notion that all operations in a transaction execute as an atomic unit. Furthermore, transactions allow multiple users to work concurrently with the same data without compromising the *consistency* of the data (consistent with database integrity rules). A particular transaction should not be visible to other concurrently running transactions; they should run in *isolation*. Changes made in a transaction should be *durable*, even if the system fails after the transaction has been completed successfully.

In addition, we want the *correctness* of a transaction. For example, the business rules dictate that the application charges the seller once, not twice. This is a reasonable assumption, but we may not be able to express it with database constraints. Hence, the correctness of a transaction is the responsibility of the application, whereas consistency is the responsibility of the database. Together, these transaction attributes define the *ACID* criteria.

11.1.2 Database and system transactions

We've also mentioned *system* and *database* transactions. Consider the last example again: during the unit of work ending an auction, we might mark the winning bid in a database system. Then, in the same unit of work, we talk to an external system to bill the seller's credit card. This is a transaction spanning several systems, with coordinated subordinate transactions on possibly several resources, such as a database connection

and an external billing processor. This chapter focuses on transactions spanning one system and one database.

Database transactions have to be short because open transactions consume database resources and potentially prevent concurrent access due to exclusive locks on data. A single database transaction usually involves only a single batch of database operations.

To execute all of the database operations inside a system transaction, we have to set the boundaries of that unit of work. We must start the transaction and, at some point, commit the changes. If an error occurs (either while executing database operations or when committing the transaction), we have to roll back the changes to leave the data in a consistent state. This process defines a *transaction demarcation* and, depending on the technique we use, involves manually defining transaction boundaries in the code. In general, transaction boundaries that begin and end a transaction can be set either programmatically in the application code or declaratively. We'll demonstrate both ways, focusing on declarative transactions while working with Spring and Spring Data.

NOTE All examples in this chapter work in any Java SE environment, without a special runtime container. Hence, from now on you'll see programmatic transaction demarcation code until we move on to specific Spring application examples.

Next we'll focus on the most complex aspect of ACID properties: how you can *isolate* concurrently running units of work from each other.

11.2 Controlling concurrent access

Databases (and other transactional systems) attempt to ensure transaction *isolation*, meaning that, from the point of view of each concurrent transaction, it appears that no other transactions are in progress. Traditionally, database systems have implemented isolation with locking. A transaction can place a lock on a particular item of data in the database, temporarily preventing read and/or write access to that item by other transactions. Some modern database engines implement transaction isolation with multi-version concurrency control (MVCC), which vendors generally consider more scalable. We'll analyze isolation assuming a locking model, but most of our observations are also applicable to MVCC.

How databases implement concurrency control is of the utmost importance in the Java Persistence application. Applications may inherit the isolation guarantees provided by the database management system, but frameworks may come on top of them and allow you to start, commit, and roll back transactions in a resource-agnostic way. If you consider the many years of experience that database vendors have with implementing concurrency control, you'll see the advantage of this approach.

Additionally, some features in Java Persistence can improve the isolation guarantee beyond what the database provides, either because you explicitly use the features or by design.

We'll discuss concurrency control in several steps. First we'll explore the lowest layer: the transaction isolation guarantees provided by the database. After that, you'll see the Java Persistence features for pessimistic and optimistic concurrency control at the application level, and other isolation guarantees that Hibernate can provide.

11.2.1 Understanding database-level concurrency

When we talk about isolation, you can assume that two transactions are either isolated or not. When we talk about database transactions, complete isolation comes at a high price. You can't stop the world in order to access data exclusively in a multiuser online transaction processing (OLTP) system. Therefore, several isolation levels are available, which, naturally, weaken full isolation but increase the performance and scalability of the system.

TRANSACTION ISOLATION PROBLEMS

First, let's examine several problems that may occur when you weaken full transaction isolation. The ANSI SQL standard defines the standard transaction isolation levels in terms of which of these phenomena are permissible.

A *lost update* occurs when two concurrent transactions simultaneously update the same information in a database. The first transaction reads a value. The second transaction starts shortly after and reads the same value. The first transaction changes and writes the updated value, and the second transaction overwrites that value with its own update. Thus, the update of the first transaction is lost, being overwritten by the second transaction. *The last commit wins*. This occurs in systems that don't implement concurrency control, where concurrent transactions aren't isolated. This is shown in figure 11.1. The `buyNowPrice` field is updated from two transactions, but only one update occurred, the other update was lost.

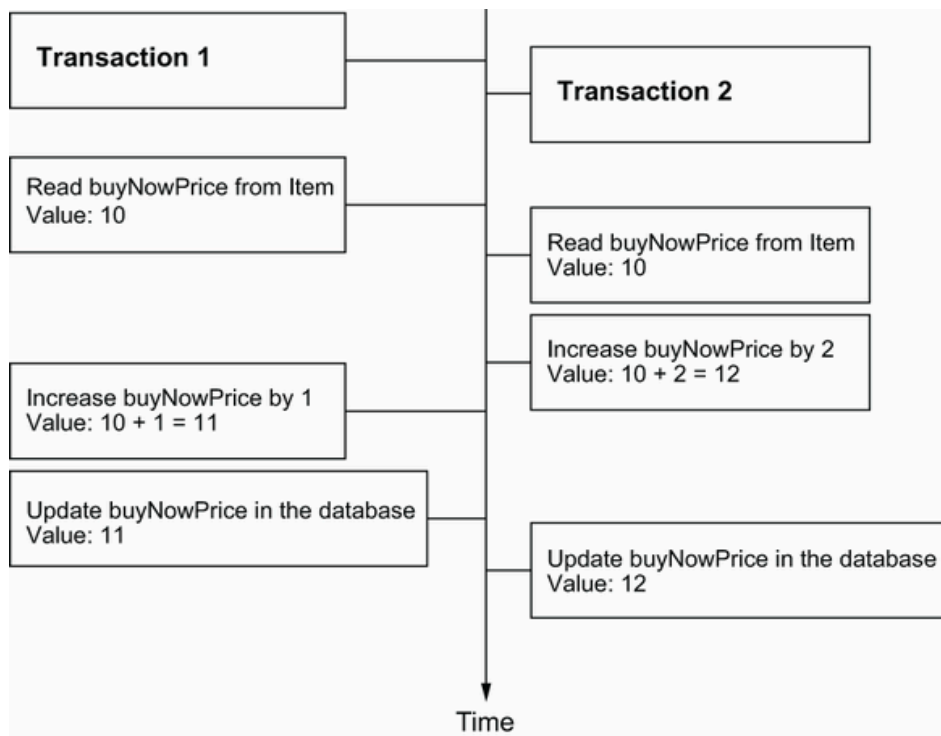


Figure 11.1 Lost update: Two transactions update the same data without isolation.

A *dirty read* occurs if transaction 2 reads changes made by transaction 1, and it hasn't yet been committed. This is dangerous because the changes made by transaction 1 may later be rolled back, and invalid data will have been read by transaction 2. This is illustrated in figure 11.2.

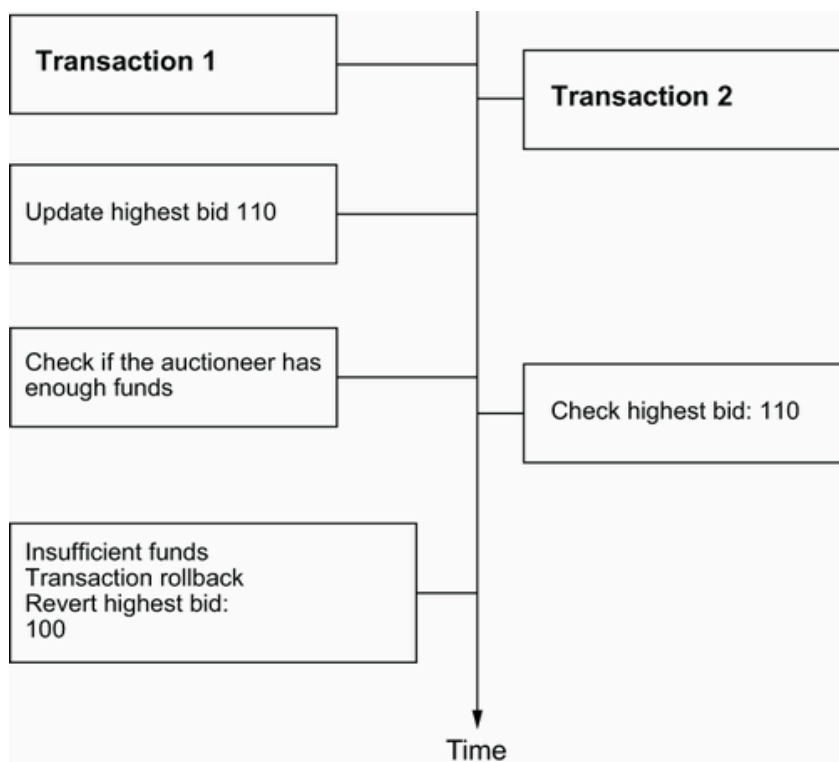


Figure 11.2 Dirty read: Transaction 2 reads uncommitted data from Transaction 1.

An *unrepeatable read* occurs if a transaction reads a data item twice and reads different states each time. For example, another transaction may have written to the data item and committed between the two reads, as shown in figure 11.3.

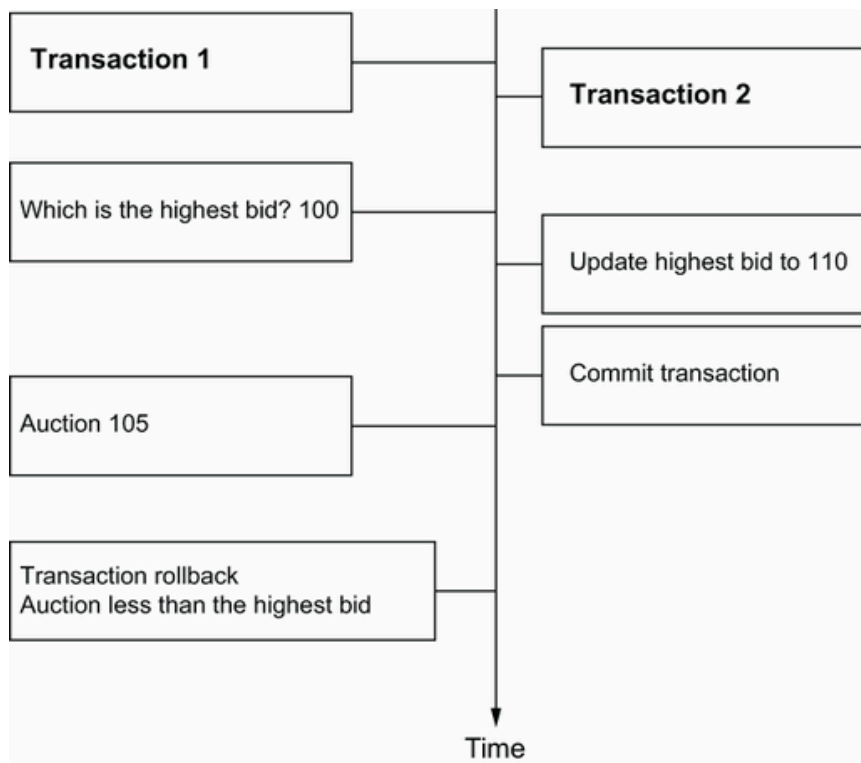


Figure 11.3 Unrepeatable read: The highest bid changed while Transaction 1 was in execution.

A *phantom read* is said to occur when a transaction executes a query twice, and the second result includes data that wasn't visible in the first result because something was added, or it includes less data because something was deleted. This need not necessarily refer to the exact same query. Another transaction inserting or deleting data between the executions of the two queries causes this situation, as shown in figure 11.4.

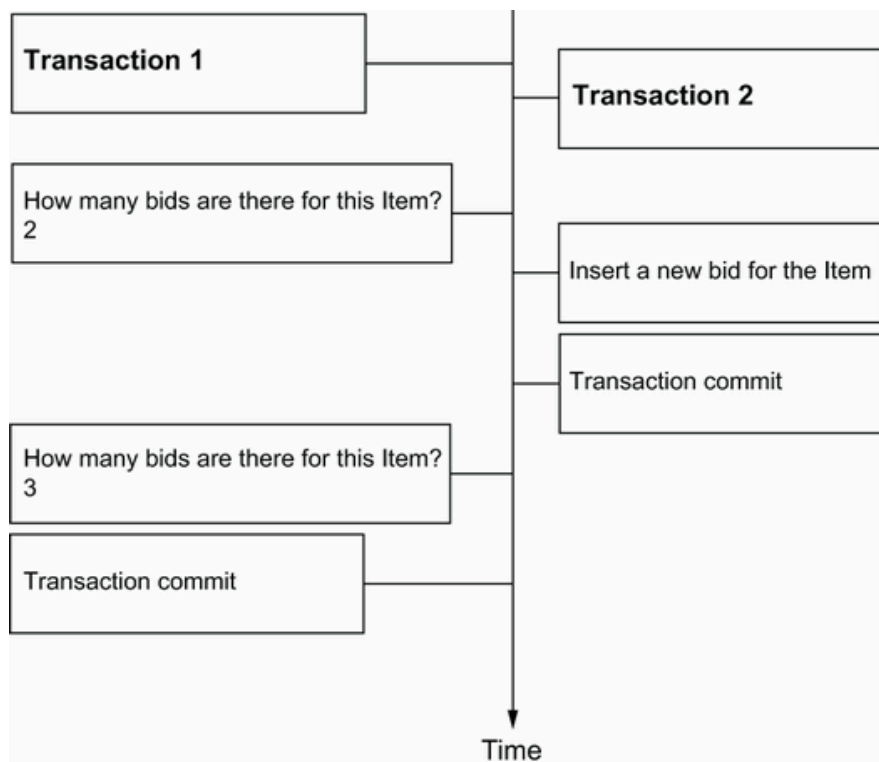


Figure 11.4 Phantom read: Transaction 1 reads new data in the second query.

Now that you understand all the bad things that can occur, we can define the transaction isolation levels and see what problems they prevent.

ANSI ISOLATION LEVELS

The standard isolation levels are defined by the ANSI SQL standard, but they aren't specific to SQL databases. Spring defines exactly the same isolation levels, and we'll use these levels to declare the desired transaction isolation. With increased levels of isolation come higher costs and serious degradation of performance and scalability:

- *Read uncommitted isolation*—A system that does not permit lost updates operates in read uncommitted isolation. One transaction may not write to a row if another uncommitted transaction has already written to it. Any transaction may read any row, however. A DBMS may implement this isolation level with exclusive write locks.
- *Read committed isolation*—A system that permits unrepeatable reads and phantom reads but neither lost updates nor dirty reads implements read committed isolation. A DBMS may achieve this by using shared read locks and exclusive write locks. Read transactions don't block other transactions from accessing a row, but an uncommitted write transaction blocks all other transactions from accessing the row.
- *Repeatable read isolation*—A system operating in repeatable read isolation mode does not permit lost updates, dirty reads, or unrepeatable reads. Phantom reads may occur. Read transactions block write transactions but do not block other read transactions, and write transactions block all other transactions.
- *Serializable isolation*—The strictest isolation, serializable, emulates serial execution as if transactions were executed one after another, rather than concurrently. A DBMS may not implement serializable isolation using only row-level locks. A DBMS must instead provide some other mechanism that prevents a newly inserted row from becoming visible to a transaction that has already executed a query that would return the row. A crude mechanism is exclusively locking the entire database table after a write so that no phantom reads can occur.

Table 11.1 summarizes the ANSI isolation levels and the problems that they address.

Table 11.1 ANSI isolation levels and the problems that they address

Isolation level	Phantom read	Unrepeatable read	Dirty read	Lost update
READ_UNCOMMITTED	–	–	–	+
READ_COMMITTED	–	–	+	+
REPEATABLE_READ	–	+	+	+
SERIALIZABLE	+	+	+	+

How exactly a DBMS implements its locking system varies significantly; each vendor has a different strategy. You should study the documentation of your DBMS to find out more about its locking system, how locks are escalated (from row-level to pages to entire tables, for example), and what effect each isolation level has on the performance and scalability of the system.

It's nice to know how all these technical terms are defined, but how does that help us choose an isolation level for the application?

CHOOSING AN ISOLATION LEVEL

Developers (ourselves included) are often unsure what transaction isolation level to use in a production application. Too high an isolation level harms the scalability of a highly concurrent application. Insufficient isolation may cause subtle, difficult-to-reproduce bugs that we won't discover until the system is working under heavy load.

Note that we'll refer to *optimistic locking* (with versioning) in the following explanation, which is a concept analyzed later in this chapter. You may want to revisit this section when it's time to pick an isolation level for your application. Choosing the correct isolation level is, after all, highly dependent on the particular scenario. The following discussion should be read as recommendations, not dictums carved in stone.

Hibernate tries hard to be as transparent as possible regarding the transactional semantics of the database. Nevertheless, persistence context caching and versioning affect these semantics. What is a sensible database isolation level to choose in a JPA application?

First, for almost all scenarios, eliminate the *read uncommitted* isolation level. It's extremely dangerous to allow one transaction's uncommitted changes to be used in a different transaction. The rollback or failure of one transaction will affect other concurrent transactions. The rollback of the first transaction could bring other transactions down with it, or per-

haps even cause them to leave the database in an incorrect state (the seller of an auction item might be charged twice—consistent with database integrity rules but incorrect). It's possible that changes made by a transaction that ends up being rolled back could be committed anyway because they could be read and then propagated by another successful transaction! You can use the *read uncommitted* isolation level for debugging purposes, to follow the execution of long insert queries, make some rough estimates of aggregate functions (such as `SUM ( * )` or `COUNT ( * )`).

Second, most applications don't need *serializable* isolation. Phantom reads aren't usually problematic, and this isolation level tends to scale poorly. Few existing applications use serializable isolation in production, but rather rely on selectively applied pessimistic locks that effectively force a serialized execution of operations in certain situations.

Next, let's consider *repeatable read*. This level provides reproducibility for query result sets for the duration of a database transaction. This means we won't read committed updates from the database if we query it several times, but phantom reads are still possible: new rows might appear, and rows we thought existed might disappear if another transaction committed changes concurrently. Although we may sometimes want repeatable reads, we typically don't need them in every transaction.

The JPA specification assumes that *read committed* is the default isolation level. This means we have to deal with unrepeatable reads and phantom reads.

Let's assume we're enabling versioning of our domain model entities, which is something Hibernate can do for us automatically. The combination of the (mandatory) persistence context cache and versioning already gives us most of the nice features of repeatable read isolation. The persistence context cache ensures that the state of the entity instances loaded by one transaction is isolated from changes made by other transactions. If we retrieve the same entity instance twice in a unit of work, the second lookup will be resolved within the persistence context cache and not hit the database. Hence, our read is repeatable, and we won't see conflicting committed data. (We can still get phantom reads, though, which are typically much easier to deal with.) Additionally, versioning switches to *first commit wins*. Hence, for almost all multiuser JPA applications, *read committed* isolation for all database transactions is acceptable with enabled entity versioning.

Hibernate retains the isolation level of the database connection; it doesn't change the level. Most products default to read committed isolation, though MySQL defaults to repeatable read. There are several ways we can change either the default transaction isolation level or the settings of the current transaction.

First, we can check whether the DBMS has a global transaction isolation level setting in its proprietary configuration. If the DBMS supports the standard SQL statement `SET SESSION CHARACTERISTICS`, we can execute it to set the transaction settings of all transactions started in this particular database *session* (which means a particular connection to the database, not a Hibernate `Session`). SQL also standardizes the `SET TRANSACTION` syntax, which sets the isolation level of the current transaction. Finally, the JDBC `Connection` API offers the `setTransactionIsolation()` method, which (according to its documentation) “attempts to change the transaction isolation level for this connection.” In a Hibernate/JPA application, we can obtain a JDBC `Connection` from the native `Session` API.

Frequently, the database connections are, by default, in read committed isolation levels. From time to time, a particular unit of work in the application may require a different, usually stricter, isolation level. Instead of changing the isolation level of the entire transaction, we should use the Jakarta Persistence API to obtain additional locks on the relevant data. This fine-grained locking is more scalable in a highly concurrent application. JPA offers optimistic version checking and database-level pessimistic locking.

11.2.2 Optimistic concurrency control

Handling concurrency in an optimistic way is appropriate when concurrent modifications are rare and it’s feasible to detect conflicts late in a unit of work. JPA offers automatic version checking as an optimistic conflict-detection procedure.

The previous sections have been somewhat dry; it’s time for some code. First we’ll enable versioning, because it’s turned off by default. Most multiuser applications, especially web applications, should rely on versioning for any concurrently modified `@Entity` instances, enabling the more user-friendly *first commit wins*.

After enabling automatic version checking, we’ll see how manual version checking works and when we have to use it.

NOTE To be able to execute the examples from the source code, you’ll first need to run the `Ch11.sql` script.

ENABLING VERSIONING

We can enable versioning with an `@Version` annotation on a special additional property of the entity class, as demonstrated next.

Listing 11.1 Enabling versioning on a mapped entity

```
Path: Ch11/transactions/src/main/java/com/manning/javapersistence/ch11
➡ /concurrency/Item.java
```

```
@Entity
public class Item {
    @Version
    private long version;
    // . . .
}
```

In this example, each entity instance carries a numeric version. It's mapped to an additional column of the `ITEM` database table; as usual, the column name defaults to the property name, here `VERSION`. The actual names of the property and column don't matter—we could rename it if `VERSION` is a reserved keyword in the DBMS.

We could add a `getVersion()` method to the class, but we shouldn't have a setter method, and the application shouldn't modify the value. Hibernate automatically changes the version value: it increments the version number whenever an `Item` instance has been found to be dirty during the flushing of the persistence context. The version is a simple counter without any useful semantic value beyond concurrency control. We can use an `int`, an `Integer`, a `short`, a `Short`, or a `Long` instead of a `long`; Hibernate wraps and starts from zero again if the version number reaches the limit of the data type.

After incrementing the version number of a detected dirty `Item` during flushing, Hibernate compares versions when executing the `UPDATE` and `DELETE` SQL statements. For example, assume that in a unit of work we load an `Item` and change its name, as follows.

Listing 11.2 Hibernate incrementing and checking the version automatically

```
Path: /Ch11/transactions/src/test/java/com/manning/javapersistence/ch11
➡ /concurrency/Versioning.java — firstCommitWins()
```

```
EntityManager em1 = emf.createEntityManager();
em1.getTransaction().begin();
Item item = em1.find(Item.class, ITEM_ID);
// select * from ITEM where ID = ?
assertEquals(0, item.getVersion());
item.setName("New Name");
// . . . Another transaction changes the record
assertThrows(OptimisticLockException.class, () -> em1.flush());
// update ITEM set NAME = ?, VERSION = 1 where ID = ? and VERSION = 0
```

Ⓐ Retrieving an entity instance by identifier loads the current version from the database with a `SELECT`.

Ⓑ The current version of the `Item` instance is `0`.

Ⓒ When the persistence context is flushed, Hibernate detects the dirty `Item` instance and increments its version to `1`. SQL `UPDATE` now performs the version check, storing the new version in the database, but only if the database version is still `0`.

Pay attention to the SQL statements, in particular, the `UPDATE` and its `WHERE` clause. This update will be successful only if there is a row with `VERSION = 0` in the database. JDBC returns the number of updated rows to Hibernate; if that result is zero, it means the `ITEM` row is either gone or doesn't have version `0` anymore. Hibernate detects this conflict during flushing, and a `javax.persistence.OptimisticLockException` is thrown.

Now imagine two users executing this unit of work at the same time, as shown previously in figure 11.1. The first user to commit will update the name of the `Item` and flush the incremented version `1` to the database. The second user's flush (and commit) will fail because their `UPDATE` statement can't find the row in the database with version `0`. The database version is `1`. Hence, the *first commit wins*, and we can catch the `OptimisticLockException` and handle it specifically. For example, we could show the following message to the second user: "The data you have been working with has been modified by someone else. Please start your unit of work again with fresh data. Click the Restart button to proceed."

What modifications trigger the increment of an entity's version?

Hibernate increments the version whenever an entity instance is dirty. This includes all dirty value-typed properties of the entity, no matter whether they're single-valued (like a `String` or `int` property), embedded (like an `Address`), or collections. The exceptions are `@OneToMany` and `@ManyToMany` association collections that have been made read-only with `mappedBy`. Adding or removing elements in these collections doesn't increment the version number of the owning entity instance. You should know that none of this is standardized in JPA—don't rely on two JPA providers to implement the same rules when accessing a shared database.

If we don't want to increment the version of the entity instance when a particular property's value has changed, we can annotate the property with `@org.hibernate.annotations.OptimisticLock(excluded = true)`.

You may not like the additional `VERSION` column in the database schema. Alternatively, you may already have a "last updated" timestamp property on the entity class and a matching database column. Hibernate can check versions with timestamps instead of using the extra counter field.

If several applications access the database, and they don't all use Hibernate's versioning algorithm, we'll have concurrency problems. An easy solution is to use database-level triggers and stored procedures: an `INSTEAD OF` trigger can execute a stored procedure when any `UPDATE` is made; it runs instead of the update. In the procedure, we can check whether the application incremented the version of the row; if the version isn't updated or the version column isn't included in the update, we know the statement wasn't sent by a Hibernate application. We can then increment the version in the procedure before applying the `UPDATE`.

VERSIONING WITH TIMESTAMPS

If the database schema already contains a timestamp column such as `LASTUPDATED` or `MODIFIED_ON`, we can map it for automatic version checking instead of using a numeric counter.

Listing 11.3 Enabling versioning with timestamps

```
Path: Ch11/transactions2/src/main/java/com/manning/javapersistence/ch11
➡ /timestamp/Item.java

@Entity
public class Item {
    @Version
    // Optional: @org.hibernate.annotations.Type(type = "dbtimestamp")
    private LocalDateTime lastUpdated;
    // . . .
}
```

This example maps the `LASTUPDATED` column to a `java.time.LocalDateTime` property; a `Date` or `Calendar` type would also work with Hibernate. The JPA standard doesn't define these types for version properties; JPA only considers `java.sql.Timestamp` portable. This is less attractive, because we'd have to import that JDBC class in the domain model. We should try to keep implementation details such as JDBC out of the domain model classes so they can be tested, instantiated, serialized, and deserialized in as many environments as possible.

In theory, versioning with a timestamp is slightly less safe, because two concurrent transactions may both load and update the same `Item` in the same millisecond; this is exacerbated by the fact that a JVM usually doesn't have millisecond accuracy (you should check your JVM and operating system documentation for the guaranteed precision). Furthermore, retrieving the current time from the JVM isn't necessarily safe in a clustered environment, where the system time of nodes may not be synchro-

nized, or time synchronization isn't as accurate as you'd need for your transactional load.

You can switch to retrieving the current time from the database machine by placing an `@org.hibernate.annotations.Type(type="dbtimestamp")` annotation on the version property. Hibernate now asks the database for the current time before updating, which gives a single source of time for synchronization. Not all Hibernate SQL dialects support this, so check the source of the configured dialect. In addition, there is always the overhead of hitting the database for every increment.

We recommend that new projects rely on versioning with a numeric counter, not timestamps. If you're working with a legacy database schema or existing Java classes, it may be impossible to introduce a version or timestamp property and column. If that's the case, Hibernate provides an alternative strategy.

VERSIONING WITHOUT VERSION NUMBERS OR TIMESTAMPS

If you don't have version or timestamp columns, Hibernate can still perform automatic versioning. This alternative implementation of versioning checks the current database state against the unmodified values of persistent properties at the time Hibernate retrieved the entity instance (or the last time the persistence context was flushed).

You can enable this functionality with the proprietary Hibernate annotation `@org.hibernate.annotations.OptimisticLocking`:

```
Path: Ch11/transactions3/src/main/java/com/manning/javapersistence/ch11
➡ /versionall/Item.java

@Entity
@org.hibernate.annotations.OptimisticLocking(
    type = org.hibernate.annotations.OptimisticLockType.ALL)
@org.hibernate.annotations.DynamicUpdate
public class Item {
    // . . .
}
```

For this strategy, you also have to enable dynamic SQL generation of UPDATE statements, using `@org.hibernate.annotations.DynamicUpdate`, as explained in section 5.3.2.

Hibernate now executes the following SQL to flush a modification of an `Item` instance:

```
update ITEM set NAME = 'New Name'
where ID = 123
and NAME = 'Old Name'
```

```

and PRICE = '9.99'
and DESCRIPTION = 'Some item for auction'
and ...
and SELLER_ID = 45

```

Hibernate lists all columns and their last known values in the `WHERE` clause. If any concurrent transaction has modified any of these values or even deleted the row, this statement returns with zero updated rows. Hibernate then throws an exception at flush time.

Alternatively, Hibernate includes only the modified properties in the restriction (only `NAME`, in this example) if you switch to `OptimisticLockType.DIRTY`. This means two units of work can modify the same `Item` concurrently, and Hibernate detects a conflict only if they both modify the same value-typed property (or a foreign key value). The `WHERE` clause of the last SQL excerpt would be reduced to `where ID = 123 and NAME = 'Old Name'`. Someone else could concurrently modify the price, and Hibernate wouldn't detect any conflict. Only if the application modified the name concurrently would we get a `javax.persistence.OptimisticLockException`.

In most cases, checking only dirty properties isn't a good strategy for business entities. It's probably not OK to change the price of an item if the description changes! This strategy also doesn't work with detached entities and merging: if we merge a detached entity into a new persistence context, the "old" values aren't known. The detached entity instance will have to carry a version number or timestamp for optimistic concurrency control.

Automatic versioning in Java Persistence prevents lost updates when two concurrent transactions try to commit modifications on the same piece of data. Versioning can also help us obtain additional isolation guarantees manually when we need them.

MANUAL VERSION CHECKING

Here's a scenario that requires repeatable database reads: imagine there are some categories in the auction system and that each `Item` is in a `Category`. This is a regular `@ManyToOne` mapping of an `Item#category` entity association.

Let's say you want to sum up all item prices in several categories. This requires a query for all items in each category, to add up the prices. The problem is, what happens if someone moves an `Item` from one `Category` to another `Category` while you're still querying and iterating through all the categories and items? With read-committed isolation, the same `Item` might show up twice while your procedure runs!

To make the “get items in each category” reads repeatable, JPA’s `Query` interface has a `setLockMode()` method. Look at the procedure in the following listing.

Listing 11.4 Requesting a version check at flush time to ensure repeatable reads

```
Path: /Ch11/transactions/src/test/java/com/manning/javapersistence/ch11
➡ /concurrency/Versioning.java — manualVersionChecking()

EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

BigDecimal totalPrice = BigDecimal.ZERO;
for (Long categoryId : CATEGORIES) {
    List<Item> items =                                     ①
        em.createQuery("select i from Item i where i.category.id = :catId",
            ➡ Item.class)
            .setLockMode(LockModeType.OPTIMISTIC)
            .setParameter("catId", categoryId)
            .getResultList();
    for (Item item : items)
        totalPrice = totalPrice.add(item.getBuyNowPrice());
}
    em.getTransaction().commit();                         ②
em.close();
assertEquals("108.00", totalPrice.toString());
```

① For each `Category`, query all `Item` instances with an `OPTIMISTIC` lock mode. Hibernate now knows it has to check each `Item` at flush time.

② For each `Item` loaded earlier with the locking query, Hibernate executes a `SELECT` during flushing. It checks whether the database version of each `ITEM` row is still the same as when it was loaded. If any `ITEM` row has a different version or the row no longer exists, an `OptimisticLockException` is thrown.

Don’t be confused by the *locking* terminology: The JPA specification leaves open how exactly each `LockModeType` is implemented. For `OPTIMISTIC`, Hibernate performs version checking; there are no actual locks involved. We’ll have to enable versioning on the `Item` entity class as explained earlier; otherwise, we can’t use the optimistic `LockModeType`s with Hibernate.

Hibernate doesn’t batch or otherwise optimize the `SELECT` statements for manual version checking; if we sum up 100 items, we get 100 additional queries at flush time. A pessimistic approach, as we’ll demonstrate later in this chapter, may be a better solution for this particular case.

Why can't the persistence context cache prevent the concurrent modification problem?

The “get all items in a particular category” query returns item data in a `ResultSet`. Hibernate then looks at the primary key values in this data and first tries to resolve the rest of the details of each `Item` in the persistence context cache—it checks whether an `Item` instance has already been loaded with that identifier.

This cache, however, doesn't help in the example procedure: if a concurrent transaction moved an item to another category, that item might be returned several times in different `ResultSet`s. Hibernate will perform its persistence context lookup and say, “Oh, I've already loaded that `Item` instance; let's use what we already have in memory.” Hibernate isn't even aware that the category assigned to the item changed or that the item appeared again in a different result.

Hence, this is a case where the repeatable-read feature of the persistence context hides concurrently committed data. We need to manually check the versions to find out if the data changed while we were expecting it not to change.

As shown in the preceding example, the `Query` interface accepts a `LockModeType`. Explicit lock modes are also supported by the `TypedQuery` and `NamedQuery` interfaces, with the same `setLockMode()` method.

An additional optimistic lock mode is available in JPA, forcing an increment of an entity's version.

FORCING A VERSION INCREMENT

What happens if two users place a bid for the same auction item at the same time? When a user makes a new bid, the application must do two things:

1. Retrieve the currently highest `Bid` for the `Item` from the database.
2. Compare the new `Bid` with the highest `Bid`; if the new `Bid` is higher, it must be stored in the database.

There is the potential for a race condition between these two steps. If, between reading the highest `Bid` and placing the new `Bid`, another `Bid` is made, you won't see it. This conflict isn't visible, and even enabling versioning of the `Item` doesn't help. The `Item` is never modified during the procedure. However, forcing a version increment of the `Item` makes the conflict detectable.

Listing 11.5 Forcing a version increment of an entity instance

```
Path: /Ch11/transactions/src/test/java/com/manning/javapersistence/ch11
➡ /concurrency/Versioning.java - forceIncrement()
```

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
Item item = em.find(                                     ❸
    Item.class,
    ITEM_ID,
    LockModeType.OPTIMISTIC_FORCE_INCREMENT
);
Bid highestBid = queryHighestBid(em, item);
// . . . Another transaction changes the record
Bid newBid = new Bid(
    new BigDecimal("45.45"),
    item,
    highestBid);
em.persist(newBid);                                     ❹
assertThrows(RollbackException.class,
    () -> em.getTransaction().commit());               ❺
em.close();
```

❸ `find()` accepts a `LockModeType`. The `OPTIMISTIC_FORCE_INCREMENT` mode tells Hibernate that the version of the retrieved `Item` should be incremented after loading, even if it's never modified in the unit of work.

❹ The code persists a new `Bid` instance; this doesn't affect any values of the `Item` instance. A new row is inserted into the `BID` table. Hibernate wouldn't detect concurrently made bids without a forced version increment of the `Item`.

❺ When flushing the persistence context, Hibernate executes an `INSERT` for the new `Bid` and forces an `UPDATE` of the `Item` with a version check. If someone modified the `Item` concurrently or placed a `Bid` concurrently with this procedure, Hibernate throws an exception.

For the auction system, placing bids concurrently is certainly a frequent operation. Incrementing a version manually is useful in many situations where we insert or modify data and want the version of some root instance of an aggregate to be incremented.

Note that if instead of a `Bid#item` entity association with `@ManyToOne`, we have an `@ElementCollection` of `Item#bids`, adding a `Bid` to the collection *will* increment the `Item` version. The forced increment then isn't necessary. You may want to review the discussion of parent/child ambiguity and how aggregates and composition work with ORM in section 8.3.

So far, we've focused on optimistic concurrency control: we expect that concurrent modifications are rare, so we don't prevent concurrent access

and detect conflicts late. Sometimes, however, we know that conflicts will happen frequently, and we want to place an exclusive lock on some data. This calls for a pessimistic approach.

11.2.3 Explicit pessimistic locking

Let's repeat the procedure demonstrated in the previous "Manual version checking" section, but this time with a pessimistic lock instead of optimistic version checking. We'll again summarize the total price of all items in several categories. This is the same code as shown earlier in listing 11.5, with a different `LockModeType`.

Listing 11.6 Locking data pessimistically

```
Path: /Ch11/transactions/src/test/java/com/manning/javapersistence/ch11
➡ /concurrency/Locking.java — pessimisticReadWrite()

EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

BigDecimal totalPrice = BigDecimal.ZERO;
for (Long categoryId : CATEGORIES) {
    List<Item> items =                                     ❶
        em.createQuery("select i from Item i where i.category.id = :catId"
            ➡ Item.class)
            .setLockMode(LockModeType.PESSIMISTIC_READ)
            .setHint("javax.persistence.lock.timeout", 5000)
            .setParameter("catId", categoryId)
            .getResultList();
    for (Item item : items)                                ❷
        totalPrice = totalPrice.add(item.getBuyNowPrice());
    // . . . Another transaction tries to obtain a lock and fails
}
em.getTransaction().commit();                             ❸
em.close();
assertEquals(0, totalPrice.compareTo(new BigDecimal("108")));
```

❶ For each `Category`, query all `Item` instances in `PESSIMISTIC_READ` lock mode. Hibernate locks the rows in the database with the SQL query. If another transaction holds a conflicting lock, wait 5 seconds if possible. If the lock can't be obtained, the query throws an exception.

❷ If the query returns successfully, we know that we hold an exclusive lock on the data, and no other transaction can access it with an exclusive lock or modify it until this transaction commits.

❸ The locks are released after committing, when the transaction completes.

The JPA specification defines that the `PESSIMISTIC_READ` lock mode guarantees repeatable reads. JPA also standardizes the `PESSIMISTIC_WRITE` mode, with additional guarantees: in addition to repeatable reads, the JPA provider must serialize data access, and no phantom reads can occur.

It's up to the JPA provider to implement these requirements. For both modes, Hibernate appends a `FOR UPDATE` clause to the SQL query when loading data. This places a lock on the rows at the database level. What kind of lock Hibernate uses depends on the `LockModeType` and the Hibernate database dialect:

- On H2 the query is `SELECT * FROM ITEM ... FOR UPDATE`. Because H2 supports only one type of exclusive lock, Hibernate generates the same SQL for all pessimistic modes.
- PostgreSQL, on the other hand, supports shared read locks: the `PESSIMISTIC_READ` mode appends `FOR SHARE` to the SQL query. `PESSIMISTIC_WRITE` uses an exclusive write lock with `FOR UPDATE`.
- On MySQL, `PESSIMISTIC_READ` translates to `LOCK IN SHARE MODE`, and `PESSIMISTIC_WRITE` to `FOR UPDATE`.

Check your database dialect. The lock is configured with the `getReadLockString()` and `getWriteLockString()` methods.

The duration of a pessimistic lock in JPA is a single database transaction. This means we can't use an exclusive lock to block concurrent access for longer than a single database transaction. When the database lock can't be obtained, an exception is thrown.

Compare this with an optimistic approach, where Hibernate throws an exception at commit time, not when you query. With a pessimistic strategy, we know that we can read and write the data safely as soon as the locking query succeeds. With an optimistic approach, we hope for the best and may be surprised later, when we commit.

Offline locks

Pessimistic database locks are held only for a single transaction. Other lock implementations are possible: for example, a lock held in memory, or a so-called *lock table* in the database. A common name for these kinds of locks is *offline locks*.

Locking pessimistically for longer than a single database transaction is usually a performance bottleneck: every data access involves additional lock checks to a globally synchronized lock manager. Optimistic locking, however, is the perfect concurrency control strategy for long-running conversations (as you'll see in the next chapter), and it performs well. Depending on the conflict-resolution strategy—which determines what

happens after a conflict is detected—the application’s users may be just as happy with optimistic locking as with blocked concurrent access. They may also appreciate the application not locking them out of particular screens while others look at the same data.

We can configure how long the database will wait to obtain the lock and block the query in milliseconds with the `javax.persistence.lock.-timeout` hint. As usual with hints, Hibernate might ignore it, depending on the database product. H2, for example, doesn’t support lock timeouts for specific queries, only a global lock timeout for the connection (defaulting to 1 second). With some dialects, such as PostgreSQL and Oracle, a lock timeout of 0 appends the `NOWAIT` clause to the SQL string.

We’ve demonstrated the lock timeout hint applied to a `Query`. We can also set the timeout hint for `find()` operations:

```
Path: /Ch11/transactions/src/test/java/com/manning/javapersistence/ch11
➡ /concurrency/Locking.java — findLock()
```

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();

Map<String, Object> hints = new HashMap<>();
hints.put("javax.persistence.lock.timeout", 5000);

Category category =
    em.find(
        Category.class,
        CATEGORY_ID,
        LockModeType.PESSIMISTIC_WRITE,
        Hints
    );

category.setName("New Name");

em.getTransaction().commit();
em.close();
```

Ⓐ Executes `SELECT ... FOR UPDATE WAIT 5000`, if supported by the dialect.

When a lock can’t be obtained, Hibernate throws either a `javax.persistence.LockTimeoutException` or a `javax.persistence.PessimisticLockException`. If Hibernate throws a `PessimisticLockException`, the transaction must be rolled back, and the unit of work ends. A timeout exception, on the other hand, isn’t fatal for the transaction. Which exception Hibernate throws again depends on the SQL dialect. For example, because H2 doesn’t support per-statement lock timeouts, we always get a `PessimisticLockException`.

We can use both the `PESSIMISTIC_READ` and `PESSIMISTIC_WRITE` lock modes even if we haven't enabled entity versioning. They translate to SQL statements with database-level locks.

The special `PESSIMISTIC_FORCE_INCREMENT` mode requires versioned entities. In Hibernate, this mode executes a `FOR UPDATE NOWAIT` lock (or whatever the dialect supports; check its `getForUpdateNowaitString()` implementation). Then, immediately after the query returns, Hibernate increments the version and makes an `UPDATE` for each returned entity instance. This indicates to any concurrent transaction that we have updated these rows, even if we haven't so far modified any data. This mode is rarely useful, except for aggregate locking as discussed in the "Forcing a version increment" section earlier in this chapter.

What about lock modes `READ` and `WRITE`?

These are older lock modes from JPA 1.0, and you should no longer use them. `LockModeType.READ` is equivalent to `OPTIMISTIC`, and `LockModeType.WRITE` is equivalent to `OPTIMISTIC_FORCE_INCREMENT`.

If we enable pessimistic locking, Hibernate locks only rows that correspond to entity instance state. In other words, if we lock an `Item` instance, Hibernate will lock its row in the `ITEM` table. If we have a joined inheritance mapping strategy, Hibernate will recognize this and lock the appropriate rows in supertable and subtables. This also applies to any secondary table mappings of an entity. Because Hibernate locks entire rows, any relationship where the foreign key is in that row will also effectively be locked: the `Item#seller` association is locked if the `SELLER_ID` foreign key column is in the `ITEM` table, but the actual `seller` instance isn't locked! Neither are collections or other associations of the `Item` where the foreign keys are in other tables.

Extending lock scope

JPA 2.0 defines the `PessimisticLockScope.EXTENDED` option. It can be set as a query hint with `javax.persistence.lock.scope`. If enabled, the persistence engine expands the scope of locked data to include any data in collections and association join tables of locked entities.

With exclusive locking in the DBMS, you may experience transaction failures because you run into deadlock situations. Let's look at how you can avoid that.

11.2.4 Avoiding deadlocks

Deadlocks can occur if the DBMS relies on exclusive locks to implement transaction isolation. Consider the following unit of work, updating two

Item entity instances in a particular order:

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
Item itemOne = em.find(Item.class, ITEM_ONE_ID);
itemOne.setName("First new name");
Item itemTwo = em.find(Item.class, ITEM_TWO_ID);
itemTwo.setName("Second new name");
em.getTransaction().commit();
em.close();
```

Hibernate executes two SQL `UPDATE` statements when the persistence context is flushed. The first `UPDATE` locks the row representing `Item` one, and the second `UPDATE` locks `Item` two:

```
update ITEM set ... where ID = 1;      ①
update ITEM set ... where ID = 2;      ②
```

① Locks row 1

② Attempts to lock row 2

A deadlock may (or may not!) occur if a similar procedure, with the opposite order of `Item` updates, executes in a concurrent transaction:

```
update ITEM set ... where ID = 2;      ①
update ITEM set ... where ID = 1;      ②
```

① Locks row 2

② Attempts to lock row 1

With a deadlock, both transactions are blocked and can't move forward, each waiting for a lock to be released. The chance of a deadlock is usually small, but in highly concurrent applications, two Hibernate applications may execute this kind of interleaved update. Note that we may not see deadlocks during testing (unless we write the right kinds of tests).

Deadlocks can suddenly appear when the application has to handle a high transaction load in production. Usually the DBMS terminates one of the deadlocked transactions after a timeout period and this transaction fails; the other transaction can then proceed. Alternatively, the DBMS may detect a deadlock situation automatically and immediately abort one of the transactions.

You should try to avoid transaction failures because they're difficult to recover from in application code. One solution is to run the database connection in *serializable* mode when updating a single row will lock the en-

ture table. The concurrent transaction has to wait until the first transaction completes its work. Alternatively, the first transaction can obtain an exclusive lock on all data when you `SELECT` the data, as demonstrated in the previous section. Then any concurrent transaction also has to wait until these locks are released.

An alternative pragmatic optimization that significantly reduces the probability of deadlocks is to order the `UPDATE` statements by primary key value: Hibernate should always update the row with primary key 1 before updating row 2, no matter in what order the data was loaded and modified by the application. You can enable this optimization for the entire persistence unit with the `hibernate.order_updates` configuration property. Hibernate then orders all `UPDATE` statements it executes in ascending order by the primary key value of the modified entity instances and collection elements detected during flushing. (As mentioned earlier, make sure you fully understand the transactional and locking behavior of your DBMS product. Hibernate inherits most of its transaction guarantees from the DBMS; for example, your MVCC database product may avoid read locks but probably depends on exclusive locks for writer isolation, and you may see deadlocks.)

We didn't have an opportunity to mention the `EntityManager#lock()` method. It accepts an already-loaded persistent entity instance and a lock mode. It performs the same locking you've seen with `find()` and a `Query`, except that it doesn't load the instance. Additionally, if a versioned entity is being locked pessimistically, the `lock()` method performs an immediate version check on the database and potentially throws an `OptimisticLockException`. If the database representation is no longer present, Hibernate throws an `EntityNotFoundException`. Finally, the `EntityManager#refresh()` method also accepts a lock mode, with the same semantics.

We've now covered concurrency control at the lowest level—the database—and the optimistic and pessimistic locking features of JPA. We still have one more aspect of concurrency to examine: accessing data outside of a transaction.

11.3 Non-transactional data access

A `JDBC Connection` is by default in *auto-commit* mode. This mode is useful for executing ad hoc SQL.

Imagine that you connect to a database with an SQL console, and you run a few queries, and maybe even update and delete rows. This interactive data access is ad hoc; most of the time you don't have a plan or a sequence of statements that you consider a unit of work. The default auto-commit mode on the database connection is perfect for this kind of data

access—after all, you don’t want to type `begin transaction` and `end transaction` for every SQL statement you write and execute.

In auto-commit mode, a (short) database transaction begins and ends for each SQL statement you send to the database. You’re effectively working in the non-transactional mode because there are no atomicity or isolation guarantees for your session with the SQL console. (The only guarantee is that a single SQL statement is atomic.)

An application, by definition, always executes a planned sequence of statements. It seems reasonable that you can therefore always create transaction boundaries to group the statements into units that are atomic and isolated from each other. In JPA, however, special behavior is associated with auto-commit mode, and you may need it to implement long-running conversations. You can access the database in auto-commit mode and read data.

11.3.1 Reading data in auto-commit mode

Consider the following example, which loads an `Item` instance, changes its `name`, and then rolls back that change by refreshing.

No transaction is active when we create the `EntityManager`. The persistence context will be in a special unsynchronized mode; Hibernate won’t flush automatically. You can access the database to read data, and such an operation executes a `SELECT`, which is sent to the database in auto-commit mode.

Usually Hibernate flushes the persistence context when you execute a `Query`. If the context is unsynchronized, flushing doesn’t occur and the query returns the old, original database value. Queries with scalar results aren’t repeatable: you see whatever values are in the database and given to Hibernate in the `ResultSet`. This also isn’t a repeatable read if you’re in synchronized mode.

Retrieving a managed entity instance involves a lookup during JDBC result-set marshaling in the current persistence context. The already-loaded instance with the changed name is returned from the persistence context; values from the database are ignored. This is a repeatable read of an entity instance, even without a system transaction.

If you try to flush the persistence context manually to store a new `Item#name`, Hibernate throws a `javax.persistence.TransactionRequiredException`. You can’t execute an `UPDATE` in unsynchronized mode, because you wouldn’t be able to roll back the change.

You can roll back the change you made with the `refresh()` method. It loads the current `Item` state from the database and overwrites the

change you made in memory.

Listing 11.7 Reading data in auto-commit mode

```
Path: Ch11/transactions4/src/test/java/com/manning/javapersistence/ch11
➡ /concurrency/NonTransactional.java

EntityManager em = emf.createEntityManager();                                ①
Item item = em.find(Item.class, ITEM_ID);                                  ②
item.setName("New Name");
assertEquals(                                                                ③
    "Original Name",
    em.createQuery("select i.name from Item i where i.id = :id", String.class)
        .setParameter("id", ITEM_ID).getSingleResult()
);
assertEquals(                                                                ④
    "New Name",
    em.createQuery("select i from Item i where i.id = :id". Item.class)
        .setParameter("id", ITEM_ID).getSingleResult().getName()
);
// em.flush();                                                                ⑤
em.refresh(item);                                                            ⑥
assertEquals("Original Name", item.getName());
em.close();
```

① No transaction is active when creating the `EntityManager`.

② Access the database to read data.

③ Because the context is unsynchronized, flushing doesn't occur, and the query returns the old, original database value.

④ The already-loaded `Item` instance with the changed name is returned from the persistence context; values from the database are ignored.

⑤ You cannot execute an `UPDATE` in unsynchronized mode, because you wouldn't be able to roll back the change.

⑥ Roll back the change you made with the `refresh()` method.

With an unsynchronized persistence context, you read data in auto-commit mode with `find()`, `getReference()`, `refresh()`, or queries. You can load data on demand as well: proxies are initialized if you access them, and collections are loaded if you start iterating through their elements. But if you try to flush the persistence context or lock data with anything but `LockModeType.NONE`, a `TransactionRequiredException` will occur.

So far, the auto-commit mode doesn't seem very useful. Indeed, many developers often rely on auto-commit for the wrong reasons:

- Many small per-statement database transactions (that's what auto-commit means) won't improve the performance of the application.
- You won't improve the scalability of the application. You might think that a longer-running database transaction, instead of many small transactions for every SQL statement, may hold database locks for a longer time, but this is a minor concern, because Hibernate writes to the database as late as possible within a transaction (flush at commit), so the database already holds write locks for a short time.
- Auto-commit provides weaker isolation guarantees if the application modifies data concurrently. Repeatable reads based on read locks are impossible with auto-commit mode. (The persistence context cache helps here, naturally.)
- If your DBMS has MVCC (for example, Oracle or PostgreSQL), you'll likely want to use its capability for *snapshot isolation* to avoid unrepeatable and phantom reads. Each transaction gets its own snapshot of the data; you only see a (database-internal) version of the data as it was before your transaction started. With auto-commit mode, snapshot isolation makes no sense, because there is no transaction scope.
- Your code will be more difficult to understand if you use auto-commit. Any reader of your code will have to pay special attention to whether a persistence context is joined with a transaction, or if it's in unsynchronized mode. If you always group operations within a system transaction, even if you only read data, everyone can follow this simple rule, and the likelihood of difficult-to-find concurrency problems is reduced.

So what are the benefits of an unsynchronized persistence context? If flushing doesn't happen automatically, you can prepare and *queue* modifications outside of a transaction.

11.3.2 Queuing modifications

The following example stores a new `Item` instance with an unsynchronized `EntityManager`.

You can call `persist()` to save a transient entity instance with an unsynchronized persistence context. Hibernate only fetches a new identifier value, typically by calling a database sequence, and assigns it to the instance. The instance will be in a persistent state in the context, but the SQL `INSERT` hasn't happened. Note that this is only possible with *pre-insert* identifier generators; see section 5.2.5.

When you're ready to store the changes, you must join the persistence context with a transaction. Synchronization and flushing occur as usual when the transaction commits. Hibernate writes all queued operations to the database.

Path: Ch11/transactions4/src/test/java/com/manning/javapersistence/ch11
➡ /concurrency/NonTransactional.java

```
EntityManager em = emf.createEntityManager();
Item newItem = new Item("New Item");
em.persist(newItem);                                ❶
assertNotNull(newItem.getId());
em.getTransaction().begin();                        ❷
if (!em.isJoinedToTransaction()) {
    em.joinTransaction();
}
em.getTransaction().commit();                       ❸
em.close();
```

❶ Call `persist()` to save a transient entity instance with an unsynchronized persistence context.

❷ Join the persistence context with a transaction.

❸ Synchronization and flushing occur when the transaction commits.

Merged changes of a detached entity instance can also be queued:

Path: Ch11/transactions4/src/test/java/com/manning/javapersistence/ch11
➡ /concurrency/NonTransactional.java

```
detachedItem.setName("New Name");
EntityManager em = emf.createEntityManager();
Item mergedItem = em.merge(detachedItem);           ❶
em.getTransaction().begin();
em.joinTransaction();
em.getTransaction().commit();                       ❷
em.close();
```

❶ Hibernate executes a `SELECT` in auto-commit mode when you `merge()`.

❷ Hibernate defers the `UPDATE` until a joined transaction commits.

Queuing also works for the removal of entity instances and `DELETE` operations:

Path: Ch11/transactions4/src/test/java/com/manning/javapersistence/ch11
➡ /concurrency/NonTransactional.java

```
EntityManager em = emf.createEntityManager();
Item item = em.find(Item.class, ITEM_ID);
em.remove(item);
em.getTransaction().begin();
em.joinTransaction();
```

```
em.getTransaction().commit();
em.close();
```

An unsynchronized persistence context, therefore, allows you to decouple persistence operations from transactions. The ability to queue data modifications, independent of transactions (and client/server requests), is a major feature of the persistence context.

Hibernate’s MANUAL flush mode

Hibernate offers a `Session#setFlushMode()` method, with the additional `FlushMode .MANUAL`. It’s a much more convenient switch that disables any automatic flushing of the persistence context, even when a joined transaction commits. With this mode, you have to call `flush()` explicitly to synchronize with the database. In JPA, the idea was that a “transaction commit should always write any outstanding changes,” so reading was separated from writing with the *unsynchronized* mode. If you don’t agree with this or don’t want auto-committed statements, enable manual flushing with the `Session` API. You can then have regular transaction boundaries for all units of work, with repeatable reads and even snapshot isolation from your MVCC database, but still queue changes in the persistence context for later execution and manually `flush()` before your transaction commits.

11.4 Managing transactions with Spring and Spring Data

We’ll now move on and demonstrate how to implement transactions with Spring and Spring Data. The transactional model used by Spring is applicable to different APIs, such as Hibernate, JPA, and Spring Data JPA. The management of the transactions can be either programmatic (as we have already demonstrated) or declarative, with the help of annotations (which is what we’ll mostly use for this section of the chapter).

The key Spring transaction abstraction is defined by the `org.springframework.transaction.PlatformTransactionManager` interface.

```
public interface PlatformTransactionManager extends TransactionManager {
    TransactionStatus getTransaction(
        ➡ @Nullable TransactionDefinition definition)
        throws TransactionException;
    void commit(TransactionStatus status) throws TransactionException;
    void rollback(TransactionStatus status) throws TransactionException;
}
```

Typically, this interface is not used directly. You will either mark transactions declaratively, through annotations, or you may eventually use `TransactionTemplate` for the programmatic transaction definition.

Spring uses the previously discussed ANSI isolation levels. For a refresher, take a look back at section 11.2.1 and particularly table 11.1, which summarizes the isolation levels and the problems that they address.

11.4.1 Transaction propagation

Spring deals with the transaction propagation problem. In brief, if `method-A` is transactional and it calls `method-B`, how will the latter behave, from the transactional point of view? Take a look at figure 11.5:

1. `bean-1` contains `method-A`, which is transactional, executed in transaction `TX1`.
2. `method-A` calls `bean-2.method-B()`, which is also transactional.

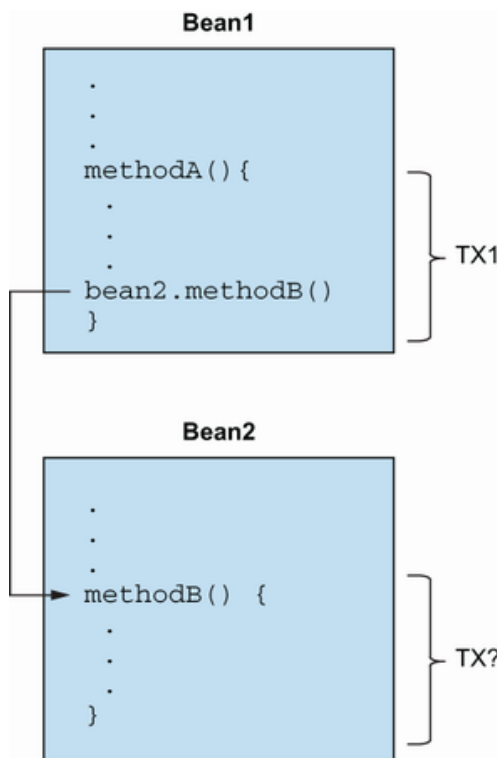


Figure 11.5 The transaction propagation concept

In which transaction will be `method-B` executed?

Spring defines the list of possible propagations through the `org.springframework.transaction.annotation.Propagation` enum:

- **REQUIRED** —If a transaction is in progress, the execution will continue within that transaction. Otherwise, a new transaction will be created. **REQUIRED** is the default propagation for transactions in Spring.
- **SUPPORTS** —If a transaction is in progress, the execution will continue within that transaction. Otherwise, no transaction will be created.

- **MANDATORY** —If a transaction is in progress, the execution will continue within that transaction. Otherwise, a `TransactionRequiredException` exception will be thrown.
- **REQUIRES_NEW** —If a transaction is in progress, it will be suspended and a new transaction will be started. Otherwise, a new transaction will be created anyway.
- **NOT_SUPPORTED** —If a transaction is in progress, it will be suspended and a non-transactional execution will continue. Otherwise, the execution will simply continue.
- **NEVER** —If a transaction is in progress, an `IllegalTransactionStateException` will be thrown. Otherwise, the execution will simply continue.
- **NESTED** —If a transaction is in progress, a subtransaction of this one will be created, and at the same time a savepoint will be created. If the subtransaction fails, the execution will roll back to this savepoint. If no transaction was originally in progress, a new transaction will be created.

Table 11.2 summarizes the possible transaction propagations in Spring (T1 and T2 are transactions 1 and 2).

Table 11.2 Transaction propagation in Spring

Transaction propagation	Transaction in the caller method	Transaction in the called method
REQUIRED	No	T1
T1	T1	SUPPORTS
No	No	T1
T1	MANDATORY	No
Exception	T1	T1
REQUIRES_NEW	No	T1
T1	T2	NOT_SUPPORTED
No	No	T1
No	NEVER	No
No	T1	Exception
NESTED	No	T1
T1	T2 with savepoint	

11.4.2 Transaction rollback

Spring transactions define the default rollback rules: a transaction will be rolled back for `RuntimeException`. This behavior can be overwritten and we can specify which exceptions automatically roll back the transaction and which will not. This is done with the help of the `@Transactional` annotation properties `rollbackFor`, `rollbackForClassName`, `noRollbackFor`, `noRollbackForClassName`. The behavior determined by these properties is summarized in table 11.3.

Table 11.3 Transaction rollback rules

Property	Type	Behavior
<code>rollbackFor</code>	Array of <code>Class</code> objects extending <code>Throwable</code>	Defines exception classes that must cause rollback
<code>rollbackForClassName</code>	Array of class names extending <code>Throwable</code>	Defines exception class names that must cause rollback
<code>noRollbackFor</code>	Array of <code>Class</code> objects extending <code>Throwable</code>	Defines exception classes that must not cause rollback
<code>noRollbackForClassName</code>	Array of class names extending <code>Throwable</code>	Defines exception class names that must not cause rollback

11.4.3 Transaction properties

The `@Transactional` annotation defines the properties in table 11.4. We'll deal here with the already examined isolation and propagation and with other properties. All the meta-information will be transposed at the level of how the transactional operation is executed.

Table 11.4 @Transactional annotation properties

Property	Type	Behavior
isolation	Isolation enum	Declares the isolation levels, following the ANSI standard.
propagation	Propagation enum	Propagation settings following the values from table 11.2.
timeout	int (seconds)	Timeout after which the transaction will automatically roll back.
readOnly	boolean	Declares if the transaction is read-only or read-write. Read-only transactions allow optimizations that can make them faster.

The `@Transactional` annotation can be applied to interfaces, to methods in interfaces, to classes, or to methods in classes. Once applied to an interface or to a class, the annotation is taken over by all methods from that class or from that interface. You can change the behavior by annotating particular methods in a different way. Also, once it's applied to an interface or to a method in an interface, the annotation is taken over by the classes implementing that interface or by the corresponding methods from the classes implementing that interface. The behavior can be overwritten. Consequently, for fine-grained behavior, it is advisable to apply the `@Transactional` annotation to methods from classes.

11.4.4 Programmatic transaction definition

Declarative transaction management is generally the way to go when using Spring in an application. It requires less code to write, and the behavior is determined through the meta-information provided by the annotations. However, programmatic transaction management is still possible, using the `TransactionTemplate` class.

Once a `TransactionTemplate` object is created, the behavior of the transaction can be defined programmatically, as follows:

```
TransactionTemplate transactionTemplate;
// . . .
transactionTemplate.setIsolationLevel(
```

```

        TransactionDefinition.ISOLATION_REPEATABLE_READ);
transactionTemplate.setPropagationBehavior(
        TransactionDefinition.PROPGATION_REQUIRES_NEW);
transactionTemplate.setTimeout(5);
transactionTemplate.setReadOnly(false);

```

Once defined, a `TransactionTemplate` object supports the callback approach through the `execute` method, which receives a `TransactionCallback` as an argument, as in the following code. The operations to be executed in the transaction are defined in the `doInTransaction` method.

```

transactionTemplate.execute(new TransactionCallback() {
    public Object doInTransaction(TransactionStatus status) {
        //operations to be executed in transaction
    }
});

```

As `TransactionCallback` is a functional interface (it even bears the `@FunctionalInterface` annotation), the preceding snippet could be shortened as follows:

```

transactionTemplate.execute(status -> {
    // operations to be executed in transaction
});

```

11.4.5 Transactional development with Spring and Spring Data

We have been working on the `CaveatEmptor` application, and now we're going to implement a feature that logs the results of our actions when we work with items. We'll start the implementation using Spring Data JPA, and we'll first create the `ItemRepositoryCustom` interface and its methods, as shown in listing 11.8. Such an interface is known as a *fragment interface*, and its purpose is to extend a repository with custom functionality, which will be provided by a later implementation.

Listing 11.8 The `ItemRepositoryCustom` interface

```

Path: Ch11/transactions5-springdata/src/main/java/com/manning
➡ /javapersistence/ch11/repositories/ItemRepositoryCustom.java

public interface ItemRepositoryCustom {
    void addItem(String name, LocalDate creationDate);
    void checkNameDuplicate(String name);
    void addLogs();
    void showLogs();
    void addItemNoRollback(String name, LocalDate creationDate);
}

```

Next, we'll create an `ItemRepository` interface, extending both `JpaRepository` and the previously declared `ItemRepositoryCustom` interface. Additionally, we'll declare the `findByName` method, following the Spring Data JPA naming conventions.

Listing 11.9 The `ItemRepository` interface

```
Path: Ch11/transactions5-springdata/src/main/java/com/manning
➡ /javapersistence/ch11/repositories/ItemRepository.java

public interface ItemRepository extends JpaRepository<Item, Long>,
    ItemRepositoryCustom {
    Optional<Item> findByName(String name);
}
```

We'll then create the `LogRepositoryCustom` interface and its methods, as shown in listing 11.10. Again, this is a fragment interface, and its purpose is to extend a repository with custom functionality that will be provided by a later implementation.

Listing 11.10 The `LogRepositoryCustom` interface

```
Path: Ch11/transactions5-springdata/src/main/java/com/manning
➡ /javapersistence/ch11/repositories/LogRepositoryCustom.java

public interface LogRepositoryCustom {
    void log(String message);
    void showLogs();
    void addSeparateLogsNotSupported();
    void addSeparateLogsSupports();
}
```

We'll now create the `LogRepository` interface, extending both `JpaRepository` and the previously declared `LogRepositoryCustom` interface.

Listing 11.11 The `LogRepository` interface

```
Path: Ch11/transactions5-springdata/src/main/java/com/manning
➡ /javapersistence/ch11/repositories/LogRepository.java

public interface LogRepository extends JpaRepository<Log, Integer>,
    LogRepositoryCustom {
}
```

We'll provide an implementation class for `ItemRepository` next. The key part of this class name is the `Impl` ending. It is not connected to Spring Data and it only implements `ItemRepositoryCustom`. When in-

jecting an `ItemRepository` bean, Spring Data will have to create a proxy class; it will detect that `ItemRepository` implements `ItemRepositoryCustom` and will look up a class called `ItemRepositoryImpl` to act as a custom repository implementation. Consequently, the methods of the injected `ItemRepository` bean will have the same behavior as the methods of the `ItemRepositoryImpl` class.

Listing 11.12 The `ItemRepositoryImpl` class

Path: `Ch11/transactions5-springdata/src/main/java/com/manning`
➡ `/javapersistence/ch11/repositories/ItemRepositoryImpl.java`

```
public class ItemRepositoryImpl implements ItemRepositoryCustom {

    @Autowired
    private ItemRepository itemRepository;

    @Autowired
    private LogRepository logRepository;

    @Override
    @Transactional(propagation = Propagation.MANDATORY)
    public void checkNameDuplicate(String name) {
        if (itemRepository.findAll().stream().map(item ->
            item.getName()).filter(n -> n.equals(name)).count() > 0) {
            throw new DuplicateItemNameException("Item with name " + name
                + " already exists");
        }
    }

    @Override
    @Transactional
    public void addItem(String name, LocalDate creationDate) {
        logRepository.log("adding item with name " + name);
        checkNameDuplicate(name);
        itemRepository.save(new Item(name, creationDate));
    }

    @Override
    @Transactional(noRollbackFor = DuplicateItemNameException.class)
    public void addItemNoRollback(String name, LocalDate creationDate) {
        logRepository.save(new Log(
            "adding log in method with no rollback for item " + name));
        checkNameDuplicate(name);
        itemRepository.save(new Item(name, creationDate));
    }

    @Override
    @Transactional
    public void addLogs() {
        logRepository.addSeparateLogsNotSupported();
    }
}
```

```

        @Override
        @Transactional
        public void showLogs() {
            logRepository.showLogs();
        }
    }
}

```

- Ⓐ Autowiring an `ItemRepository` and a `LogRepository` bean.
- Ⓑ MANDATORY propagation: Spring Data will check if a transaction is already in progress and will continue with it. Otherwise, an exception will be thrown.
- Ⓒ Throw a `DuplicateItemNameException` if an `Item` with the given name already exists.
- Ⓓ Default propagation is `REQUIRED`.
- Ⓔ No rollback of the transaction in the case of a `DuplicateItemNameException`.

Next we'll provide an implementation class for `LogRepository`. As was the case for `ItemRepositoryImpl`, the key part of this class name is the `Impl` ending. It only implements `LogRepositoryCustom`. When we inject a `LogRepository` bean, Spring Data will detect that `LogRepository` implements `LogRepositoryCustom` and will look up a class called `LogRepositoryImpl` to act as a custom repository implementation. Consequently, the methods of the injected `LogRepository` bean will have the same behavior as the methods of the `LogRepositoryImpl` class.

Listing 11.13 The `LogRepositoryImpl` class

Path: `Ch11/transactions5-springdata/src/main/java/com/manning`
 ➡ `/javapersistence/ch11/repositories/LogRepositoryImpl.java`

```

public class LogRepositoryImpl implements LogRepositoryCustom {
    @Autowired
    private LogRepository logRepository;

    @Override
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void log(String message) {
        logRepository.save(new Log(message));
    }

    @Override
    @Transactional(propagation = Propagation.NOT_SUPPORTED)

```

```

        public void addSeparateLogsNotSupported() {
            logRepository.save(new Log("check from not supported 1"));
            if (true) throw new RuntimeException();
            logRepository.save(new Log("check from not supported 2"));
        }

        @Override
        @Transactional(propagation = Propagation.SUPPORTS)           ⑤
        public void addSeparateLogsSupports() {
            logRepository.save(new Log("check from supports 1"));
            if (true) throw new RuntimeException();
            logRepository.save(new Log("check from supports 2"));
        }

        @Override
        @Transactional(propagation = Propagation.NEVER)              ⑥
        public void showLogs() {
            System.out.println("Current log:");
            logRepository.findAll().forEach(System.out::println);
        }
    }
}

```

④ Autowiring a `LogRepository` bean.

⑤ `REQUIRES_NEW` propagation. Spring Data will execute the logging in a separate transaction, independent of the eventual transaction of the method that called `log`.

⑥ The `log` method will save a message to the repository.

⑦ `NOT_SUPPORTED` propagation. If a transaction is in progress, it will be suspended and a non-transactional execution will continue. Otherwise, the execution will simply continue.

⑧ `SUPPORTS` propagation. If a transaction is in progress, the execution will continue within that transaction. Otherwise, no transaction will be created.

⑨ `NEVER` propagation. If a transaction is in progress, an `IllegalTransactionStateException` will be thrown. Otherwise, the execution will simply continue.

We'll now write a series of tests to verify the behavior of the transactional methods we have just written.

Listing 11.14 The `TransactionPropagationTest` class

```

Path: Ch11/transactions5-springdata/src/test/java/com/manning
➡ /javapersistence/ch11/concurrency/TransactionPropagationTest.java

```



```

@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = {SpringDataConfiguration.class})
public class TransactionPropagationTest {

    @Autowired
    private ItemRepository itemRepository;

    @Autowired
    private LogRepository logRepository;

    @BeforeEach
    public void clean() {
        itemRepository.deleteAll();
        logRepository.deleteAll();
    }

    @Test
    public void notSupported() {
        assertAll(
            () -> assertThrows(RuntimeException.class, () ->
                itemRepository.addLogs()),
            () -> assertEquals(1, logRepository.findAll().size()),
            () -> assertEquals("check from not supported 1",
                logRepository.findAll().get(0).getMessage())
        );
        logRepository.showLogs();
    }

    @Test
    public void supports() {
        assertAll(
            () -> assertThrows(RuntimeException.class, () ->
                logRepository.addSeparateLogsSupports()),
            () -> assertEquals(1, logRepository.findAll().size()),
            () -> assertEquals("check from supports 1",
                logRepository.findAll().get(0).getMessage())
        );
        logRepository.showLogs();
    }

    @Test
    public void mandatory() {
        IllegalStateException ex =
            assertThrows(IllegalTransactionStateException.class,
                () -> itemRepository.checkNameDuplicate("Item1"));
        assertEquals("No existing transaction found for transaction mark
            with propagation 'mandatory'", ex.getMessage());
    }

    @Test
    public void never() {
        itemRepository.addItem("Item1", LocalDate.of(2022, 5, 1));
        logRepository.showLogs();

        IllegalStateException ex =

```

```

        assertThrows(IllegalTransactionStateException.class,
            () -> itemRepository.showLogs());
    assertEquals(
        "Existing transaction found for transaction marked with propagat
        'never'", ex.getMessage());
}

@Test
public void requiresNew() {
    itemRepository.addItem("Item1", LocalDate.of(2022, 5, 1));
    itemRepository.addItem("Item2", LocalDate.of(2022, 3, 1));
    itemRepository.addItem("Item3", LocalDate.of(2022, 1, 1));

    DuplicateItemNameException ex =
        assertThrows(DuplicateItemNameException.class, () ->
            itemRepository.addItem("Item2", LocalDate.of(2016, 3, 1)));
    assertEquals("Item with name Item2 already exists",
        ex.getMessage());
    assertEquals(4, logRepository.findAll().size());
    assertEquals(3, itemRepository.findAll().size());
};

System.out.println("Logs: ");
logRepository.findAll().forEach(System.out::println);

System.out.println("List of added items: ");
itemRepository.findAll().forEach(System.out::println);
}

@Test
public void noRollback() {
    itemRepository.addItemNoRollback("Item1",
        ➡ LocalDate.of(2022, 5, 1));
    itemRepository.addItemNoRollback("Item2",
        ➡ LocalDate.of(2022, 3, 1));
    itemRepository.addItemNoRollback("Item3",
        ➡ LocalDate.of(2022, 1, 1));

    DuplicateItemNameException ex =
        assertThrows(DuplicateItemNameException.class,
            () -> itemRepository.addItem("Item2",
                LocalDate.of(2016, 3, 1)));
    assertEquals("Item with name Item2 already exists",
        ex.getMessage());
    assertEquals(4, logRepository.findAll().size());
    assertEquals(3, itemRepository.findAll().size());
};

System.out.println("Logs: ");
logRepository.findAll().forEach(System.out::println);

System.out.println("List of added items: ");
itemRepository.findAll().forEach(System.out::println);
}

```

```
}  
}
```

- Ⓐ Autowiring an `ItemRepository` and a `LogRepository` bean.
- Ⓑ Before the execution of each test, all `Item` entities and all `Log` entities are removed from the repositories.
- Ⓒ The `addLogs` method starts a transaction, but it calls the `addSeparateLogsNotSupported` method, which will suspend the started transaction before explicitly throwing an exception.
- Ⓓ Before an exception was thrown, the `logRepository` was able to save one message.
- Ⓔ The `showLog` method will display one message in a non-transactional way.
- Ⓕ The `addSeparateLogsSupports` method will explicitly throw an exception.
- Ⓖ Before an exception was thrown, the `logRepository` was able to save one message.
- Ⓗ The `showLog` method will display one message in a non-transactional way.
- Ⓘ The `checkNameDuplicate` method can be executed only in a transaction, so an `IllegalTransactionStateException` will be thrown when calling it without a transaction. We also check the message from the exception.
- Ⓣ After adding an `Item` to the repository, it is safe to call the `showLogs` method from `LogRepository` without a transaction.
- Ⓚ However, we cannot call the `showLogs` method from `LogRepository` within a transaction, as the calling method `showLogs` from `ItemRepository` is transactional.
- Ⓛ Trying to insert a duplicate `Item` in the repository will throw a `DuplicateItemNameException`.
- Ⓜ However, a log message is persisted in the logs even after exception, because it was added in a separate transaction.
- Ⓝ The repository will contain 4 `Log` messages (one for each attempt to insert an `Item`, successful or unsuccessful), but only 3 `Items` (the duplicate `Item` was rejected).

Ⓢ Trying to insert a duplicate `Item` in the repository will throw a `DuplicateItemNameException`.

Ⓢ However, a log message is persisted in the logs even after the exception, because the transaction was not rolled back. The `addItemNoRollback` method from `ItemRepository` does not roll back for `DuplicateItemNameException`.

Ⓢ The repository will contain 4 `Log` messages (one for each attempt to insert an `Item`, successful or unsuccessful), but only 3 `Items` (the duplicate `Item` was rejected).

Summary

- Hibernate relies on a database's concurrency-control mechanism but provides better isolation guarantees in a transaction, thanks to automatic versioning and the persistence context cache.
- Transaction boundaries can be set programmatically, and you can handle exceptions.
- You can use optimistic concurrency control and explicit pessimistic locking.
- You can work with auto-commit mode and an unsynchronized persistence context outside of a transaction.
- You can work with transactions with Spring and Spring Data, defining and configuring transactions using various properties.